Open Sound Control (OSC)
native support in RTcmix v5.9.0

RTcmix OSC tutorial

* https://rtcmix.org/tutorials/osc.html

Definition of Open Sound Control

* https://ccrma.stanford.edu/groups/osc

AGENDA

- Introductions
- What is OSC?
- Demo
- Why OSC?
- Fun technical details (time allowing)
- Origin story (time allowing)
- Reference

David Reeder

davidreeder@mobilesound.org
http://mobilesound.org

My role is: instigator, consultant and a composer who creates RTcmix scores.
    I provide examples and rationale for using OSC in RTcmix.
        Origin story later: "The shim".   (time allowing)

Doug did the coding.

Thank you, Ryan, for helping me understand the 'RT' in RTcmix.

Original demo, proposal and workaround for OSC in RTcmix v5.0.0:

    * https://github.com/davidreeder/Python-MOSToolkit/tree/main/demos/MOSOSC-with-RTcmix

Find today's assets in directory "fest2026"

    Browse these assests or download fest2026.tgz.

    BEWARE -- NOT an official release. Provided only for today.
              To be updated after Doug and I have finalized all changes.

              But... you don't need it anyways!
              ...Unless you'd like an example to get started.
              ...Or wish to use MOSOSC as your OSC client.

What is OSC? (high level)

An OSC client

    sends named messages (+ data)

        to a server to control its behavior.

In this case, to control sound objects.

    Message data may contain parameters of any complexity.

Its a mini-protocol.

    SEE CNMAT for the specification.
    SEE ALSO doc/ directory in "fest2026".

OSC paths are heirarchical, and look exactly like UNIX paths.

Demo

Python client sends real-time OSC messages

performed by a real-human

that trigger and manage four RTcmix sound objects:

- low notes (+ ornaments)
- high notes
- "boom"
- "kapow" (The Looch, barking in latin rhythm...)

Client runs from commandline, has simple QWERTY interface.

NOTE
Current RTcmix implementation (v5.9.0) produces the same results as original demo (v5.0.0), where heavy lifting done by RTcmix, instead of "the shim". (SEE slide about origin story.)

Why OSC?

score + performance

Why OSC? (more details...)

. Layer of indirection
    - Score definition vs. performance are separate

. Realtime control

. Sometimes a live performance is better than pre-captured audio result.
    - EG: Sonifying slices of a complex database that is constantly updating.

. Enables adaptable sound objects

. Low latency: OSC in UDP over local WIFI

. Adjacent to MIDI. OSC is "MIDI 2.0".

Scope of RTcmix implementation

. MinC is not a general-purpose programming language.

  (Though it does quite a lot!)

. OSC in RTcmix naturally reflects the capabilities of MinC.

  SEE RTcmix OSC tutorial for details.

. A few canonical OSC features are missing:

    - Timestamps
    - Blobs
    - OSC Address Patterns (aka globbing in OSC paths)

    Blobs and timestamps enable coordination of multiple OSC messages
    scheduled for future execution.

    OSC Address Patterns are the OSC path equivalent of UNIX shell globbing.
    IE: Path expansion upon interpretation by the server.

. Meanwhile, a bonus feature: nested lists!

    - Doug hacked liblo to allow nested lists. This is why liblo is
      now part of RTcmix build.

    - IMHO, nested lists is one of the most important parts of OSC
      specification.

. Let's wait to see if people use the existing features... :)

Origin story: OSC emulation for RTcmix v5.0.0.

. What's happening in the OSC emulation demo (RTcmix v5.0.0)?
  What is "the shim"?

. Original OSC implementation recognized a single OSC path:
    - /RTcmix/ScoreCommands

. Turns out CMIX can incrementally update the current score via
  stdin or via "/RTcmix/ScoreCommands", which takes a string that
  appends to the active score.

. "The shim" is a combination of things:

    - Python OSC client that is RTcmix savvy: translating Python
      parameters and arrays into lists;

    - An RTcmix score function (here called "main(...)") that
      accepts lists representing OSC paths;

    - main() routes incoming arguments to local functions that
      represent sound objects.

. All this remains possible in RTcmix, but don't do it!

. As of RTcmix v5.9.0, the shim is replaced by RTcmix native OSC implementation.

REFERENCE

RTcmix OSC tutorial

* https://rtcmix.org/tutorials/osc.html

Definition of Open Sound Control (OSC)

* https://ccrma.stanford.edu/groups/osc

MOSOSC-with-RTcmix ("the shim")

* https://github.com/davidreeder/Python-MOSToolkit/tree/main/demos/MOSOSC-with-RTcmix

MOSOSC

* https://github.com/davidreeder/Python-MOSToolkit/